

The code: how to find your way around it, file by file and function by function

OCloud Solutions · `c:\mob_ai\poc` on 11 September 2026 · 56 Python files · 12,017 lines · one database · two model folders

This is the document to have open while reading or changing the code. It gives the map, the reading conventions, the entry points and what they call, every data shape that crosses a function boundary, a file-by-file reference, the database field by field, the files on disk, and a "where do I change X" table.

- A The map — every file, one line each
- B How to read this code — the conventions
- C Entry points and what they call
- D The data shapes that cross boundaries
- E The walk through `agent.run_one`, in execution order
- F File-by-file reference
- G The database, field by field
- H Files on disk that are not code
- I Environment variables and flags
- J Where do I change X
- K How to add a connector, a stage, a prompt, an action
- L Known gaps and sharp edges

A • The map

```
c:\mob_ai\poc\
|
| — entry points (things a person runs) —————
| run.py                106  first scan: every envelope into the database → report.html
| agent.py              399  the full pipeline over N newest messages, every stage printed
| watch.py              273  live: poll, run the pipeline, label, optionally draft and ask
| scan.py               154  catch up on the backlog by calling agent.run_one
| web.py                687  the mailbox in a browser; --watch runs the agent behind it
| chat.py               294  sentences → actions, through user_input and the gate
| send_mail.py          156  send on the Owner's behalf, through stages 19-11-13-14
| brief.py              244  the daily brief; --email sends it (UC-31)
| waiting.py            191  unanswered outgoing mail (UC-22)
| inbox.py              193  merged inbox, search, duplicates collapsed (UC-27/28)
| mailbox.py            295  three-tier view in the terminal; --warm fills the 30-day store
| correct.py            334  tell it it was wrong (UC-29)
| history.py            158  what it did, and --undo (UC-30)
| my_data.py            189  export or erase (UC-32)
| sort_mailbox.py       238  apply labels in bulk; --undo removes exactly those
|
| — the spine —————
| connect.py            180  picks the door; live_id / fetch_verified for safe reopen
| store.py              402  the database: schema + every query
| connectors\
|   | base.py           104  Envelope dataclass + the Connector protocol
|   | gmail.py          470  the Gmail API door (OAuth, history.list, thread-local clients)
|   | imap.py           937  the IMAP door (X-GM-EXT-1, UID commands, SMTP send)
| pipeline\
|   | stage02_strip.py   97   MIME → text; quoted history and signature removed
|   | stage03_sensitive.py 195 headers-only gate; four sources of sensitivity
|   | stage04_headers.py 91   facts from headers; the engagement exit
|   | stage05_forms.py   189  .ics, .pkpass, embedded invoice XML, layout fingerprint
|   | stage07_dates.py   219  find date phrases; resolve with code, never a model
|   | stage09_gate.py    236  needs a reply? hand weights blended with LightGBM
|   | stage10_redact.py  128  the one door out: patterns + known names → placeholders
|   | stage13_approval.py 201  the gate (Approval, check) and the executor (execute)
|   | stage15_unsubscribe.py 93  RFC 8058 one-click only
|   | stage16_commitments.py 231 date → reminder, lead time by kind
|   | stage17_quick.py   73   eight fixed replies, suggested by rules
|   | stage19_voice.py   239  writing profile + TF-IDF examples in SQLite
|   | model.py           324  Gemini client; classify (with the stage-6 router),
|   |                   pick_deadline, draft_reply, read_command
|   | local_classifier.py 117  our XLM-R classifier, loaded once; topic by rules
|   | local_pii.py        108  our XLM-R token model; 113 labels → families
|   | user_input.py       177  typed text → one of a fixed set of actions
|   | prompts.py          94   loads prompts.md
|   | batch_label.py      76   the teacher labels ten emails per request
|
| — training —————
| train_classifier.py    272  label with Gemini (distillation), then fine-tune XLM-R
| train_gate.py          176  LightGBM on this mailbox's own open/reply history
| train_pii.py           153  token model on ai4privacy/pii-masking-200k
| kaggle_train.py        226  the classifier training as a Kaggle notebook
| kaggle_train_pii.py    192  the PII training as a Kaggle notebook
| dump_for_labelling.py  97   numbered dump for hand labelling → to_label.txt
```

```

| apply_labels.py      138  labels.txt + to_label.txt → labelled.jsonl
| learn_voice.py      90   Sent folder → style profile + examples
| shrink_model.py     133  PyTorch → ONNX 8-bit (written, not yet run)
|
| — proofs —————
| test_imap.py        182  the first IMAP proof (Primary tab, same pipeline)
| audit_imap.py       567  47 capabilities, writes on probe messages, self-verifying
| verify_doc.py       308  the comparison notes, claim by claim
| limits_imap.py      279  sizes, speed, caps, connections
| runlog.py           48   tee every proof run into test_runs\
| analyze.py          91   findings as header-only queries
| report.py           173  findings → report.html
|
| — data and artefacts —————
| mailbox.db          the database (7,391 messages · 11 bodies · 12 decisions)
| classifier\         stage-8 model, 547 MB (config, safetensors, tokenizer)
| pii_model\         stage-10 model, 1.1 GB
| gate_model.txt / .json  stage-9 LightGBM model + metadata (17 features, AUC 0.684)
| prompts.md         every prompt, six sections
| applied_labels.json  every label applied, with the door that applied it
| labelled.jsonl · labels.txt · to_label.txt  classifier training data
| credentials.json · token.json              OAuth client + the user's grant
| requirements.txt
| test_runs\         one file per proof run, timestamped

```

B • How to read this code

1 • Every file opens with why it exists

The module docstring is the design note. It says what the file does, the commands that run it, and the reasoning — usually with a measured number. Read the docstring first; the code below it will then make sense.

2 • marks a decision that was not obvious

A  comment sits beside code that was written the way it is for a reason a reviewer would need: what was wrong before, how it was found, why the fix is this and not the alternative. Many name the run in `test_runs/` that found the problem. If you are about to change a line under one of these, read it first.

3 • Stage numbers are names, not an order

`stage09_gate.py` is "stage 9" everywhere: in the code, the documents and the meeting notes. The execution order differs from the numbers (3 before 2; 16 after 7; 6 after 8) and Part E gives the real order. There is no stage 18.

4 • No provider name above the connector line

Only `connect.py` and `connectors/` contain "gmail" or "imap". Everything else asks `conn.supports_*`. If you find yourself writing `if conn.name == ...` in a stage, the capability flag you need is missing from `base.py`; add it there.

5 • Two kinds of id, and which is which

`message_id` is the RFC header (looks like `<abc@mail.example.com>`) and is the real key. `provider_id` is the door's handle: a 16-character hex string from the API, or a short all-digit UID from IMAP. Never fetch by a stored `provider_id` directly; go through `connect.fetch_verified`.

6 • Reversible vs irreversible is a set, not a judgement

`stage13_approval.IRREVERSIBLE = {"send", "forward", "unsubscribe", "trash"}` and `REVERSIBLE = {"label", "archive", "mark_read", "snooze", "remind", "rescue_from_spam", "draft"}`. A new action must be added to one of them or `check()` refuses it.

7 • Print is not record

A verdict is recorded only by `store.record_decision`, an action only by `store.record_action`. Printing it is not enough; the watcher printed verdicts for weeks and recorded none. If a surface makes a

decision, it writes it.

C · Entry points and what they call

```
run.py ————— connect.open_mailbox → conn.list_ids → conn.fetch_envelopes → store.save
                  → analyze.scan → report.write (report.html)

agent.py ————— connect.open_mailbox, Store
                  for each envelope: run_one(conn, store, env, names, want_draft, audit)
                  run_one is THE pipeline. Everything else that runs the pipeline calls it.

scan.py ————— unscanned(store) → for each: connect.live_id → conn.fetch_envelopes
                  → agent.run_one (stdout captured) → one log line

watch.py ————— conn.current_history_id → loop: conn.new_since → conn.fetch_envelopes
                  → store.save → handle(...) [its own compact stage walk; records stage-9]
                  → conn.apply_label → applied_labels.json → draft_and_ask (if --draft)

web.py ————— Store, connect.open_mailbox (quiet)
GET /          render_list → rows() → classify() per row → HTML
GET /open      render_message → store.body or connect.fetch_verified → stage02.strip
GET /how       render_how → live counts from store
--watch        thread: watcher() → own conn + Store → new_since / scan.scan → agent.r

chat.py ————— loop: input → user_input.read(text, state) → one of do_brief / do_search /
                  do_sort / do_watch / do_compose / do_send / do_edit / do_discard / do_help
                  do_compose → model.draft_reply → state["approval"] = Approval("send", body)
                  do_send → approval.grant(typed) → stage13_approval.execute("send", ...)

send_mail.py — stage19 (profile + similar) → model.draft_reply → Approval → execute("send")

brief.py ————— rows → stage03/04 facts → local_classifier → stage09.score → top N
                  → model.summarise (capped at 8) → print or conn.send (--email)

correct.py — store.decision_for / add_correction; --scan: conn.label_removals_since
history.py — store.action_history; --undo: the undo map → connector call → record_action("undo")
my_data.py — store.dump → JSON; --erase: delete rows, keep the mailbox untouched
sort_mailbox.py ensure_labels → decide() per row → conn.apply_label → applied_labels.json
mailbox.py — rows_for → worth_keeping → connect.fetch_verified → store.cache_body; trim_bodies
```

The one function that is the pipeline. `agent.run_one(conn, store, env, names, want_draft, audit)`. It is called by `agent.py`, `scan.py` and the web watcher. If you change what a stage does, change it in the stage module and every caller gets it. `watch.py` has its own compact walk in `handle()` for historical reasons; it records the stage-9 decision the same way and should eventually call `run_one` too.

D • The data shapes that cross boundaries

Envelope — `connectors/base.py`

A dataclass. Required: `provider_id`, `message_id`. Then, all defaulted: `thread_id`, `account`, `sender`, `sender_name`, `sender_domain`, `recipients`, `subject`, `date`, `size`, `unread`, `in_inbox`, `categories`, `unsubscribe`, `one_click`, `list_id`, `bulk`, `auto_submitted`, `in_reply_to`, `references`. The defaulted field `account` must stay after the required ones (a dataclass cannot have a default before a required field). **No body.**

facts — from `stage04_headers.read(env)`

`{machine_generated, has_unsubscribe, one_click_unsubscribe, list_id, auto_reply, is_reply, addressed_only_to_me, auth: {checked, passed, note}}`. Booleans and one nested dict. Consumed by stages 9, 16 and the surfaces.

hist — from `store.history(address, me)`

`{sent, opened, replied}` — counts of messages from this address, how many were opened, how many we replied to. `replied` works only because the `recipients` column exists; before it was added this count was always zero and the gate's heaviest feature was permanently false.

labels — from `local_classifier.classify()` or `model.classify()`

`{intent, topic, confidence, by}` plus, when escalated, `escalated_from`, or `note` when escalation was skipped. `intent` is one of the two binary labels from our model or one of the six from the Gemini prompt; `stage09_gate.REQUESTING` lists the values that count as "a request" from *both* vocabularies.

sc — from `stage09_gate.score(labels, facts, hist, env)`

`{act_probability, needs_reply, first_contact, scored_by, signals, why}`. `needs_reply = act_probability >= 0.45 and signals["is_a_request"]`. `why` is the comma-joined list of true signals and is what gets recorded and shown.

cands — from `stage07_dates.find_candidates(text)`

A list of `{phrase, resolved (ISO), days_away, looks_like_deadline, context}`. Empty when nothing looked like a date.

forms — from `stage05_forms.extract(attachments, html)`

`{meetings: [...], passes: [...], invoices: [...], fingerprint}`.

`stage16.label_key(commitment, forms)` uses only these declared sources to pick a category label.

commitment — from `stage16_commitments.from_email(env, cands, forms, facts)`

`{due, kind, remind_at, why, source}` or `None`. `source` is "attached invite — declared, not inferred" or "found in the text".

Approval — `stage13_approval.Approval(action, draft)`

Holds `hash = sha256(draft)[:16]`, `requested_at`, `granted`. `grant(typed)` returns `(ok, reason)`; it is an exact match against `AFFIRMATIVE` and expires after 24 h. `check(action, draft, approval)` is the gate; `execute(...)` is the only place a connector's write methods are called from the pipeline.

state — in `chat.py`

`{current_draft, draft_to, draft_subject, approval}`. `user_input.allowed_actions(state)` builds the multiple-choice list from this: with no `current_draft`, "send" and "edit" are not offered, so no phrasing can produce them.

E • The walk through `agent.run_one` , in execution order

Line numbers are approximate to the current file; the order and the calls are exact.

STEP	STAGE	CALL	EXIT?
1	1	print <code>conn.name</code> and the capability flags	—
2	3	<code>stage03_sensitive.check(env.sender, env.subject, env.sender_domain)</code>	stop if sensitive; body never fetched
3	4	<code>facts = stage04_headers.read(env); hist = store.history(env.sender, me); stage04_headers.engagement_exit(hist.sent, hist.opened)</code>	exit if ≥ 8 received and 0 opened
4	2	<code>raw = conn.fetch_raw(env.provider_id); parsed = stage02_strip.strip(raw) → {text, html, attachments, ...}</code>	—
5	5	<code>forms = stage05_forms.extract(attachments, "")</code>	declared data noted
6	7	<code>cands = stage07_dates.find_candidates(parsed.text)</code>	—
7	16	<code>commitment = stage16_commitments.from_email(env, cands, forms, facts); if any, stage16_commitments.record(store, env, commitment)</code>	reminder written, no approval
8	8	<code>labels = model.classify(parsed.text, env.subject, env.sender) →</code> tries <code>local_classifier</code> first	—
9	6	report what <code>classify</code> did: kept ours / escalated (confidence < 0.65) / no key	—
10	9	<code>sc = stage09_gate.score(labels, facts, hist, env); store.record_decision(env.message_id, "9", ..., sc.why, sc.act_probability)</code>	exit if not needs_reply (unless drafting was asked for anyway)
11	—	<code>store.already_replied(env.message_id)</code>	exit if already answered
12	10	<code>redacted, mapping = stage10_redact.redact(stage10_redact.minimise(parsed.text), names);</code> sender name redacted the same way	mapping stays in memory
13	7/11	<code>deadline = model.pick_deadline(cands, env.subject)</code> if any candidates	—
14	19	profile and similar replies from <code>stage19_voice</code>	—
15	11	<code>draft = model.draft_reply(redacted, env.subject, sender_tok, ...)</code>	skipped without a key

16	12	<code>final, leftover = stage10_redact.restore(draft, mapping)</code>	discard if any placeholder unresolved
17	13 (UC-25)	with <code>--drafts : stage13_approval.execute(conn, "draft", ...)</code> → Drafts folder; <code>store.record_action(..., "draft", ...)</code>	done, reversible
18	13	<code>ap = Approval("send", final) ; show the draft; ap.grant(input())</code>	nothing unless an exact yes
19	14	<code>stage13_approval.execute(conn, "send", env.provider_id, final, ap, audit, to=..., subject=..., in_reply_to=..., references=..., thread_id=...) ; on success store.mark_replied</code>	the only send

F • File-by-file reference

Purpose, the functions that matter, what each reads and writes, and who calls it. Entry points are in Part C; this part covers the spine, the pipeline and the helpers.

connect.py

HOSTS · host_for(address)	Domain → IMAP server. Unknown domains fall back to <code>imap.<domain></code> .
want_imap()	<code>--imap</code> wins; <code>--api</code> wins; else IMAP if <code>IMAP_PASSWORD</code> is set and no <code>token.json</code> .
open_mailbox(quiet)	Returns a connector. Prints the door unless quiet.
describe(conn)	One line of capabilities for headers.
looks_imap_uid(pid)	All digits and ≤10 characters. A 16-digit hex id that happens to have no letters is not a UID.
live_id(conn, pid, mid)	The id to use on this door: <code>pid</code> on the API door for API ids; otherwise <code>conn.find_by_message_id(mid)</code> .
fetch_verified(conn, pid, mid)	Fetches by <code>live_id</code> , then checks the Message-ID inside the bytes matches <code>mid</code> . Raises <code>LookupError</code> on missing or mismatch. Use this, never <code>conn.fetch_raw(stored_pid)</code>.

store.py — class Store

<code>__init__(path="mailbox.db")</code>	Opens with <code>check_same_thread=False</code> , runs <code>SCHEMA</code> , migrates old databases by <code>ALTER TABLE</code> (appending <code>recipients</code> , <code>account</code> , <code>date_iso</code>).
<code>save(envelopes) · have()</code>	Upsert envelopes (22 columns, <code>date_iso</code> computed by <code>iso_date()</code> , which returns "" rather than a guess); the set of provider ids already held.
<code>record_decision · decision_for</code>	Write / read the <code>decisions</code> table. Stage "9" is the reply gate; "sort" is the labeller.
<code>add_correction · correction_for · all_corrections</code>	The corrections table; <code>correction_for(sender, domain)</code> returns the most recent matching correction, checked before any model is trusted.
<code>record_action · action_history · mark_undone</code>	The actions table. action_history, not history — that name is the sender-history method.
<code>cache_body · body · has_body · trim_bodies(days) · body_stats</code>	The bounded 30-day store. <code>trim_bodies</code> is called on every view open.

add_reminder · due_reminders · clear_reminder · pending_reminders	Reminders and snoozes.
already_replied · mark_replied	The <code>handled</code> table, so a second run never replies twice.
history(address, me)	<code>{sent, opened, replied}</code> for one sender. <code>replied</code> counts our messages whose <code>recipients</code> contains the address.
q(sql, *args) · one(sql, *args)	Raw query helpers used by every surface.

connectors/gmail.py — class GmailConnector

SCOPES · HEADERS	Two scopes; the thirteen header names fetched.
__init__(credentials, token, workers=6)	OAuth flow or refresh; builds the service; thread-local services per worker; <code>errors</code> list.
list_ids · fetch_envelopes · fetch_raw · fetch_body · fetch_attachment	Reading. <code>fetch_envelopes</code> uses a thread pool with <code>format=metadata</code> .
apply_label · remove_label · ensure_label · archive · mark_read · trash	Writes through <code>messages.modify</code> . <code>trash</code> never calls <code>messages.delete</code> .
send · create_draft · forward	Sending. Threading headers set so a reply lands in the conversation.
list_spam · rescue_from_spam · unsnooze	Spam is read to rescue from, never to file into.
current_history_id · new_since(history_id)	Polling via <code>history.list</code> ; 404 on a stale id resets the bookmark.
label_removals_since(history_id, our_label_ids)	UC-29's correction feed: labels the Owner removed in Gmail.
find_by_message_id(mid)	<code>messages.list(q="rfc822msgid:...")</code> .

connectors/imap.py — class ImapConnector

KNOWN · DEFAULT_FOLDERS · SMTP_HOSTS · _SPAM_CANDIDATES	Per-provider folder and server names. Gmail's are <code>[Gmail]/Sent Mail</code> , <code>[Gmail]/Trash</code> , <code>[Gmail]/Spam</code> .
---	---

<code>__init__(host, user, password)</code>	TLS login, <code>SELECT INBOX</code> , then <code>has_gmail_extensions()</code> flips the capability flags; <code>can_send()</code> sets <code>supports_send</code> .
<code>has_gmail_extensions · gmail_search(query) · gmail_labels(uid)</code>	<code>CAPABILITY</code> for <code>X-GM-EXT-1</code> ; <code>UID SEARCH X-GM-RAW</code> ; <code>X-GM-LABELS</code> parsed with a bracket-aware tokenizer (quoted or bare).
<code>list_ids · fetch_envelopes · _fetch_one · _envelope_from</code>	<code>UID SEARCH ALL</code> ; batched <code>UID FETCH</code> of 100 with <code>(UID FLAGS BODY.PEEK[HEADER])</code> , yielded in the caller's order, falling back to one at a time if a batch fails.
<code>fetch_raw · fetch_attachment</code>	<code>UID FETCH (BODY.PEEK[])</code> — never marks read.
<code>apply_label · remove_label · ensure_label · _delimiter</code>	On Gmail: <code>UID STORE ±X-GM-LABELS</code> , message stays put. Elsewhere: <code>UID COPY</code> into a folder named with the server's own separator, quoted.
<code>archive · trash · _move · _has_move</code>	Archive is <code>MOVE</code> to All Mail on Gmail (removing <code>\Inbox</code> from inside INBOX silently does nothing). Trash is a <code>MOVE</code> . Fallback without <code>MOVE</code> uses <code>UID EXPUNGE</code> scoped to one UID, never plain <code>EXPUNGE</code> .
<code>mark_read · mark_unread</code>	<code>UID STORE ±FLAGS (\Seen)</code> .
<code>list_spam · _spam_folder · rescue_from_spam</code>	Finds the spam folder by trying names; rescue selects it and issues one atomic <code>UID MOVE</code> to INBOX; refuses on a server without <code>MOVE</code> .
<code>send · create_draft · forward</code>	SMTP on 587 with <code>STARTTLS</code> ; copy APPENDED to Sent. Draft APPENDED to Drafts with <code>\Draft</code> . Forward quotes the original's plain text, never re-attaches.
<code>current_history_id · new_since</code>	Highest UID as the bookmark; <code>UID SEARCH UID n+1:*</code> with the clamp-to-highest quirk filtered out; INBOX re-selected on every poll.
<code>label_removals_since(bookmark, our_labels, footprint=None) · _labelled_by_us</code>	No feed on IMAP, so re-read <code>X-GM-LABELS</code> on the messages we labelled (from <code>applied_labels.json</code> , IMAP-door rows only) and report ours that are gone — skipping messages that are no longer in INBOX, which is not a correction.
<code>find_by_message_id(mid)</code>	<code>UID SEARCH HEADER Message-ID "<...>"</code> in the selected folder.

pipeline/model.py

<code>MODELS · ESCALATE_BELOW = 0.65 · INTENT · TOPIC</code>	The Gemini fallback order, the router threshold, the two label sets.
<code>_client() · _generate(prompt) · _json_from(text)</code>	One client per process; tries each model twice; pulls the first JSON object out of the reply.
<code>classify(text, subject, sender)</code>	Stage 8 with the stage-6 router inside: our model → keep if confident → else Gemini with <code>escalated_from</code> → else keep ours with a <code>note</code> .

<code>pick_deadline(candidates, subject)</code>	The model chooses which found phrase is the real deadline; code already resolved each one.
<code>draft_reply(redacted_text, subject, sender_placeholder, profile, examples, ...)</code>	Stage 11.
<code>read_command(user_text, state)</code>	The chat's multiple-choice interpreter; used only after <code>user_input.by_rules</code> found nothing.

pipeline/user_input.py

ALL_ACTIONS · The twelve actions; "send" is the irreversible one; word-boundary rules in priority order;
 IRREVERSIBLE · RULES refusals matched before anything else.
 · NEGATIVE

<code>allowed_actions(state)</code>	Built by our code from state; the model only ever chooses from this list.
<code>by_rules(text, allowed)</code>	Refusal → decline; an email address → compose; else the first rule that matches.
<code>read(text, state)</code>	Rules first; model second; anything outside <code>allowed</code> becomes "nothing".

The other stages, in one line each

<code>stage02_strip</code>	<code>strip(raw_bytes) → {text, html, attachments, ...}; html_to_text, pick_body.</code>
<code>stage03_sensitive</code>	<code>check(sender, subject, domain) → {sensitive, reasons, notice}; user_marked(), mark_sensitive(domain).</code> No body parameter, by design.
<code>stage04_headers</code>	<code>read(env, auth_results) → facts; read_auth; engagement_exit(sent, opened, min_messages=8).</code>
<code>stage05_forms</code>	<code>extract(attachments, html) → forms; read_calendar, read_wallet_pass, find_invoice_xml, structure_fingerprint, validate_extraction.</code>
<code>stage07_dates</code>	<code>find_candidates(text), resolve_relative, add_working_days, parse_exact, reminder_time; LOOKS_LIKE_DATE and RELATIVE patterns around dateparser.</code>
<code>stage09_gate</code>	<code>score(labels, facts, history, env) → sc; WEIGHTS, REQUESTING, TRAINED_FEATURES; _trained_score loads gate_model.txt once.</code>
<code>stage10_redact</code>	<code>minimise(text, 3000), redact(text, known_names) → (text, mapping), restore(draft, mapping) → (text, leftover); PATTERNS for nine identifier kinds.</code>

stage13_approval	Approval, is_approval, check, execute; the sets IRREVERSIBLE, REVERSIBLE, AFFIRMATIVE; APPROVAL_TTL.
stage15_unsubscribe	options(header, one_click) → {can_one_click, https, ...}, one_click(url), describe(opts).
stage16_commitments	from_email, record, classify_kind, reminder_time, label_key, learn_from_reaction, lead_for; LEAD, KIND_WORDS (order is priority), LABEL_FOR.
stage17_quick	suggest(text) → up to 3 keys, text_for(key); ANSWERS, TRIGGERS.
stage19_voice	build_profile, describe, class Embedder (TF-IDF, 384-dim), class VoiceStore (learn, profile, find_similar, count).
local_classifier · local_pii	available(), _load() once, classify(text, subject, sender) / find(text); TOPIC_RULES; FAMILY map.
prompts · batch_label	get(name, **fills), names(), reload(); label_batch(emails) ten per request.

G · The database, field by field

TABLE · KEY	COLUMNS
messages PK provider_id	message_id (the real key; indexed by sender, thread, in_reply_to) · thread_id · sender · sender_name · sender_domain · subject · date (raw RFC) · size · unread · in_inbox · categories · unsubscribe · one_click · list_id · bulk · auto_sub · in_reply_to · refs · recipients · account · date_iso (sortable). No body.
bodies PK message_id	provider_id · account · text · chars · fetched_at (indexed; what trim_bodies uses).
decisions PK (message_id, stage)	decision · reason · score · at . One row per stage per message; a re-run replaces.
actions PK id	message_id · provider_id · action · detail · before · undone · at (indexed).
corrections PK id	scope (sender · domain · message · reminder_lead) · target (lower-cased) · was · should_be · source (app · mailbox) · at . Indexed on (scope, target).
reminders PK (message_id, kind)	provider_id · kind (remind · snooze) · due (ISO; indexed with done) · subject · why · done .
handled PK (message_id, action)	sent_id · at .
style_examples · style_profile	Stage 19: text , subject , embedding BLOB ; and the profile's counted fields.

Two things to know when querying. Order by date_iso , never date . And provider_id is the primary key of messages , so the same email seen through both doors is two rows; collapse on message_id when showing (inbox.collapse_duplicates).

H • Files on disk that are not code

FILE	WRITTEN BY	READ BY	CONTENTS
mailbox.db	store.py, stage19	everything	The database.
classifier/	kaggle_train.py	local_classifier	config.json (2 labels), model.safetensors, tokenizer files. 547 MB.
pii_model/	kaggle_train_pii.py	local_pii	config.json (113 labels), model.safetensors, tokenizer. 1.1 GB.
gate_model.txt · gate_model.json	train_gate.py	stage09_gate	LightGBM model text; feature order, AUC 0.6841, 5,895/1,474 rows, trained 2026-09-05.
prompts.md	a person	prompts.py	Six prompts. <code>{braces}</code> filled by code; <code>></code> lines are notes, never sent.
applied_labels.json	sort_mailbox, watch	sort_mailbox --undo, imap.label_removals_since	[[id, door, label_id, label, subject, at], ...]
user_sensitive.json	correct.py (never open this)	stage03 on every call	Domains the Owner marked private.
labelled.jsonl	train_classifier label / apply_labels	train_classifier train, kaggle_train	199 lines: text, intent, topic.
to_label.txt · labels.txt	dump_for_labelling; a person	apply_labels	The hand-labelling round trip; numbers must match.
credentials.json · token.json	Google Cloud console; the OAuth flow	connectors/gmail	Secrets. Never commit.
report.html	report.py	a browser	The first-scan findings.
test_runs/*.txt	runlog.py	people	Verbatim output of every proof run, named <code><script>_<flags>_<timestamp>.txt</code> .

I • Environment variables and flags

NAME	READ IN	EFFECT
IMAP_USER	connect, imap, proofs	Which account; also the From address on SMTP.
IMAP_PASSWORD	connect, imap.send	The app password. Its presence (with no token.json) makes IMAP the default door.
GEMINI_API_KEY	model_client, agent, chat	Enables stage 11, escalation in stage 6, deadline picking, chat fallback, brief summaries.
--imap / --api	connect.want_imap	Force the door.
--draft / --drafts	agent, watch	Draft; draft into the Drafts folder without asking.
--every N	watch	Poll interval, seconds (default 15).
--watch / --port N	web	Background agent; port (default 8000).
--write / --send / --no-connections	audit_imap, verify_doc, limits_imap	Enable write tests; send to yourself; skip the connection probe.
python -X utf8	the Windows console	Needed so subjects with characters outside cp1252 print instead of crashing.

J • Where do I change X

I WANT TO...	CHANGE
add a bank or a sensitive domain	<code>stage03_sensitive.SENSITIVE_DOMAINS</code> , or a suffix in <code>SENSITIVE_SUFFIXES</code> ; or let the user mark it (<code>correct.py</code> → <code>user_sensitive.json</code>).
change when something "needs a reply"	<code>stage09_gate.WEIGHTS</code> and the 0.45 threshold in <code>score()</code> ; retrain with <code>train_gate.py</code> .
change the escalation threshold	<code>model.ESCALATE_BELOW</code> .
change how a draft is worded	<code>prompts.md</code> , section <code>draft_reply</code> . No code.
change reminder lead times	<code>stage16_commitments.LEAD</code> ; per-person via <code>correct.py --timing</code> .
add a quick reply	<code>stage17_quick.ANSWERS</code> and a trigger in <code>TRIGGERS</code> .
add a word that counts as "yes"	<code>stage13_approval.AFFIRMATIVE</code> . Exact match only.
add a chat command	<code>user_input.ALL_ACTIONS</code> , a rule in <code>RULES</code> , a <code>do_*</code> in <code>chat.py</code> , and <code>allowed_actions</code> if it depends on state.
change how long bodies are kept	<code>WINDOW_DAYS</code> in <code>mailbox.py</code> and <code>web.py</code> (both call <code>store.trim_bodies</code>).
change what the web page shows per row	<code>web.classify()</code> for the verdict; <code>render_list()</code> for the HTML.
add an IMAP provider	<code>connect.HOSTS</code> , <code>imap.KNOWN</code> (sent/trash names), <code>imap.SMTP_HOSTS</code> .
change the poll interval	<code>watch.py --every</code> ; <code>web.watcher(every=20)</code> .
change the batch size for IMAP fetch	<code>ImapConnector.BATCH</code> (100).
add a header to the envelope	<code>base.Envelope</code> (after the required fields), <code>gmail.HEADERS</code> , <code>imap._envelope_from</code> , <code>store.SCHEMA</code> + <code>save</code> , and the migration.

K · How to add a connector, a stage, a prompt, an action

A connector (e.g. Microsoft Graph)

1. Create `connectors/graph.py` with a class that implements every method in `base.Connector` and sets every `supports_*` flag honestly.
2. Return an `Envelope` from `fetch_envelopes` with no body. Fill `account`.
3. Implement `find_by_message_id` so `connect.fetch_verified` works.
4. Add the choice to `connect.open_mailbox()`. Nothing else in the project changes.
5. Run `audit_imap.py` 's equivalent against it before claiming anything.

A stage

1. Create `pipeline/stageNN_name.py` with a module docstring saying why it exists and what it must never see.
2. Give it a pure function that takes the shapes from Part D and returns a dict.
3. Call it from `agent.run_one` at the right point in the *execution* order, and print a `stage(NN, ...)` line.
4. If it decides something, `store.record_decision`. If it acts, go through `stage13_approval.execute`, and add the action to `REVERSIBLE` or `IRREVERSIBLE`.

A prompt

Add a `## name` section to `prompts.md`. Use `{placeholders}`. Load it with `prompts.get("name", **fills)`. Anything that reaches it from an email must have passed through `stage10_redact.redact` first.

An action the executor can take

1. Add the connector method to both connectors (or raise clearly on the one that cannot).
2. Add the name to `IRREVERSIBLE` or `REVERSIBLE` in `stage13_approval`.
3. Add its undo to the map in `execute()`, and a branch that calls the connector.
4. Add a plain-English name in `history.PLAIN`.

L • Known gaps and sharp edges

watch.py has its own stage walk	<code>handle()</code> duplicates part of <code>run_one</code> . It records stage-9 decisions correctly now, but any new stage must be added in both places until it is made to call <code>run_one</code> .
waiting.py is API-only	It calls <code>conn.svc.users().threads()</code> and the labels API directly. Needs a connector-level "thread of" and Sent-folder access to work over IMAP.
IMAP UIDs are per folder	A UID from <code>list_spam()</code> is only meaningful with the spam folder selected. <code>rescue_from_spam</code> handles this; anything new that takes a UID from another folder must too.
Old rows saved with sequence numbers	Repaired on 2026-09-11 (<code>test_runs/repair_stale_ids_*.txt</code>). Any database from before that date should be checked the same way before trusting its IMAP ids.
Python heredocs in this shell eat backslashes	Writing Python source through a bash heredoc turned <code>\n</code> into a real newline and <code>\b</code> into a byte. Use the editor, or build escapes with <code>chr(92)</code> .
The two model folders are large	547 MB + 1.1 GB. They are not in any repository; they are rebuilt from the Kaggle notebooks. <code>shrink_model.py</code> is the path to something shippable.
The sample-based numbers move	Anything measured on "the 25 newest" changes daily. The limits test weighs every message for that reason; if a figure must be defended, measure everything.
Gemini key hygiene	A key was exposed in a chat during the project. Rotate it, and never print it.